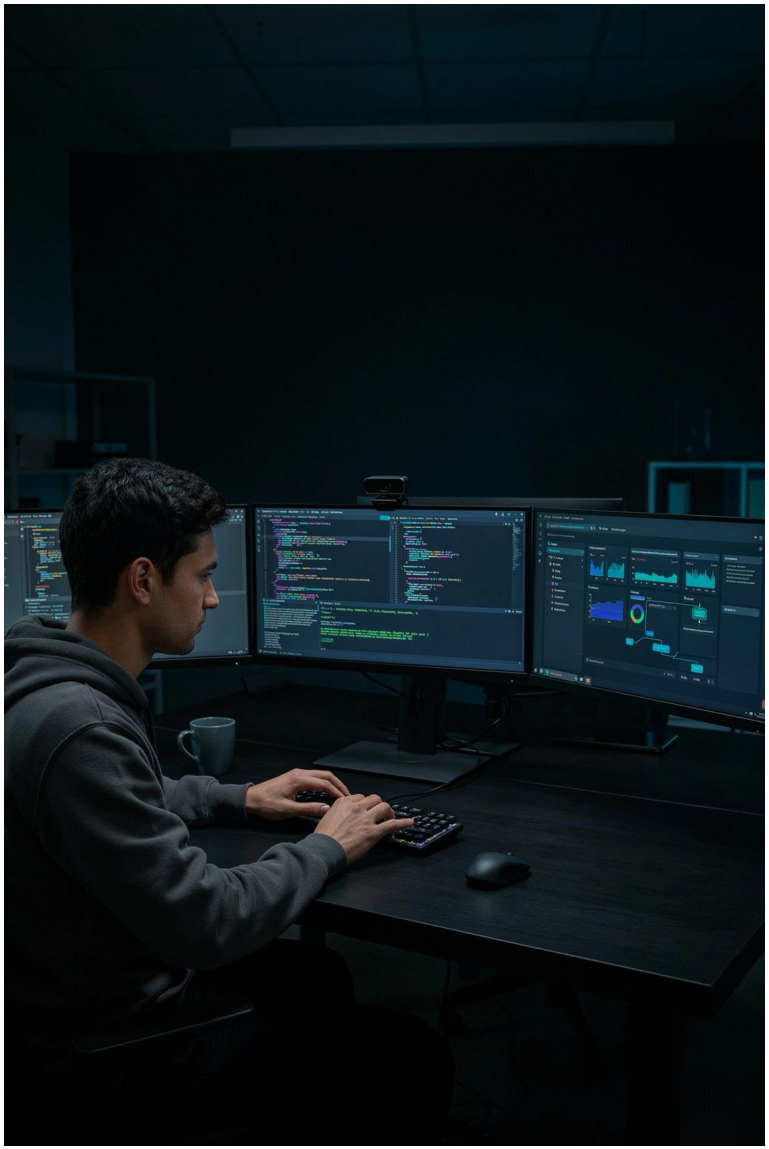
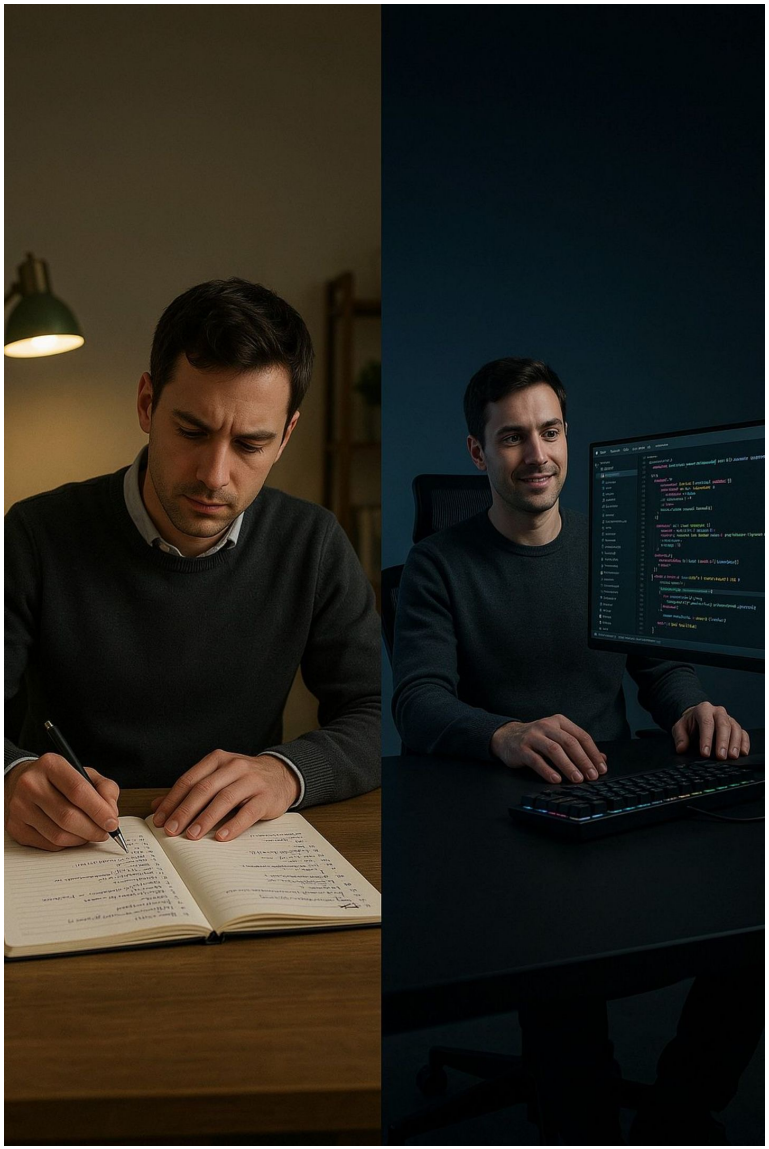
Spec-Driven Development (SDD)

Como desenvolver software com IA sem depender apenas de prompts

Vibe Coding → SDD → Agentic Engineering





A IA mudou o desenvolvimento de software

Antes

Ideia → Requisitos → Desenvolvedor → Código

Agora

Ideia → IA → Código em segundos

Mas surge um novo problema:

Se gerar código ficou fácil, como garantir que estamos gerando o código certo?

> O gargalo está saindo da escrita do código e indo para intenção, contexto e validação.

O problema não é mais gerar código

Modelos de IA conseguem:

- **Criar código**
- **Refatorar**
- **Criar testes**
- **Documentar**
- **Analisar código existente**
- **Corrigir bugs**

O desafio passa a ser:

O que exatamente queremos que a IA construa?

E:

Como sabemos que ela terminou corretamente?

O que é Vibe Coding?

Vibe Coding

Desenvolvimento guiado pela conversa e pelo resultado.

1

Prompt

Você descreve o que quer.

2

Código

A IA gera alguma coisa.

3

"Não era bem isso..."

Você olha o resultado. Pede alterações.

4

Repete

Até parecer correto.



Vibe Coding na prática

Exemplo

"Crie uma procedure Oracle para aumentar o salário de um funcionário."

A IA gera:

```
CREATE OR REPLACE PROCEDURE aumentar_salario ...
```

Você testa.

Resultado

Depois lembra: "Não pode aumentar mais que 20%."

Depois: "Precisa registrar auditoria."

Depois: "Precisa tratar employee inexistente."

Os requisitos vão aparecendo durante a implementação.

Quando Vibe Coding funciona muito bem

Vibe Coding é excelente para:



Protótipos



Exploração



Scripts pequenos



Testes de ideias



Código descartável



Aprendizado

O problema aparece quando entram:

complexidade + equipe + produção + regras de negócio + manutenção.

A própria documentação do Kiro diferencia esse desenvolvimento mais exploratório do uso de specs em funcionalidades complexas, que exigem planejamento, documentação e rastreabilidade.

Surge o Spec-Driven Development

Spec-Driven Development

Desenvolvimento guiado pela especificação.

Em vez de começar perguntando:

"Qual código devemos escrever?"

Começamos perguntando:

"Qual comportamento o sistema precisa ter?"

No SDD: **Spec → Código**

e não: **Prompt → Código**

O GitHub descreve SDD como um processo no qual a especificação define o "what" antes do "how", com refinamento em múltiplas etapas em vez de geração one-shot.

A grande mudança do SDD

No desenvolvimento tradicional:

Código é a principal fonte de verdade.

A especificação frequentemente vira documentação secundária.

No SDD:

A especificação ganha protagonismo

Ela orienta:

- Arquitetura
- Implementação
- Testes
- Critérios de aceite
- Validação
- Evolução do sistema

O fluxo do SDD

1. **SPECIFY**
   O que precisa ser construído?
2. **PLAN**
   Como será construído?
3. **TASKS**
   Quais trabalhos precisam ser executados?
4. **IMPLEMENT**
   Gerar e modificar o código
5. **CONVERGE**
   Validar até implementação e especificação convergirem

Esse é o fluxo atual documentado pelo GitHub Spec Kit.

O que existe dentro de uma boa SPEC?

Uma especificação pode definir:

Objetivo

O que queremos resolver?

Requisitos

O que o sistema deve fazer?

Regras de negócio

O que pode e o que não pode acontecer?

Restrições

Tecnologia, arquitetura, segurança, performance.

Critérios de aceite

Como sabemos que terminou?

Casos de teste

Como provar que funciona?

Quanto mais verificável a especificação, menos a IA precisa adivinhar.

Exemplo Oracle PL/SQL com Vibe Coding

Prompt

Crie uma procedure PL/SQL para aumentar o salário de um funcionário.

Problemas:

- Qual percentual?
- Existe limite?
- Quem pode executar?
- Precisamos de auditoria?
- O funcionário pode não existir?
- Pode ocorrer rollback?
- Qual mensagem deve retornar?

⚠ A IA precisa preencher as lacunas. E é exatamente nessas lacunas que aparecem muitos erros.

O mesmo problema usando SDD

SPEC: Increase Employee Salary

Objetivo

Criar uma procedure para atualização controlada de salário.

Entrada

- employee_id
- increase_percentage

Regras

- Funcionário deve existir
- Percentual deve ser entre 0 e 20%
- Nenhum COMMIT dentro da procedure
- Toda alteração deve gerar auditoria
- Erros devem ser tratados explicitamente

Critérios de aceite

- Funcionário válido é atualizado
- Funcionário inexistente gera erro
- Percentual >20% é rejeitado
- Auditoria registra valor anterior e novo valor

Agora a IA recebe uma SPEC

A conversa muda.

De:

"Crie uma procedure."

Para:

"Implemente esta especificação."

A IA pode então:

01

Analisar requisitos

02

Planejar implementação

03

Gerar PL/SQL

04

Criar testes

05

Executar testes

06

Comparar resultado com critérios de aceite

07

Corrigir divergências

A IA deixa de apenas gerar código.

Ela passa a implementar uma intenção estruturada.

Vibe Coding x SDD

Vibe Coding	Spec-Driven Development
Conversa	Especificação
Exploração	Planejamento
Prompt	Requisitos
Resultado visual	Critérios de aceite
Iteração manual	Validação estruturada
IA adivinha lacunas	Lacunas são explicitadas
Ótimo para protótipos	Melhor para sistemas complexos

Não significa abandonar Vibe Coding. Significa usar estrutura quando a complexidade exigir.

Como SDD muda o papel do desenvolvedor?

Antes, grande parte do trabalho era:

Escrever código

Com IA + SDD, cresce a importância de:

- **Definir intenção**
- **Definir restrições**
- **Projetar arquitetura**
- **Criar critérios de aceite**
- **Validar resultados**
- **Gerenciar contexto**

O desenvolvedor sobe um nível de abstração.

E onde entra Agentic Engineering?

Agentic Engineering

Engenharia de sistemas em que agentes recebem:

Contexto

Código, documentação, arquitetura e histórico.

Ferramentas

Terminal, Git, banco de dados, APIs, MCP.

Regras

Padrões, segurança, arquitetura e limites.

Objetivos

Specs e tarefas.

Validação

Testes, evals, lint, CI/CD e revisões.

Feedback loops

Errou → analisa → corrige → testa novamente.

Sistemas agentic modernos dependem justamente dessa combinação de contexto, ferramentas, loops de execução e mecanismos de avaliação. Anthropic descreve agentes de forma simples como LLMs usando ferramentas autonomamente em loop; experiências recentes da OpenAI mostram agentes trabalhando com ferramentas, testes, revisões e feedback até concluir tarefas.

SDD x Agentic Engineering

SDD responde:

O QUE construir?

- Spec
- Requisitos
- Regras
- Critérios de aceite

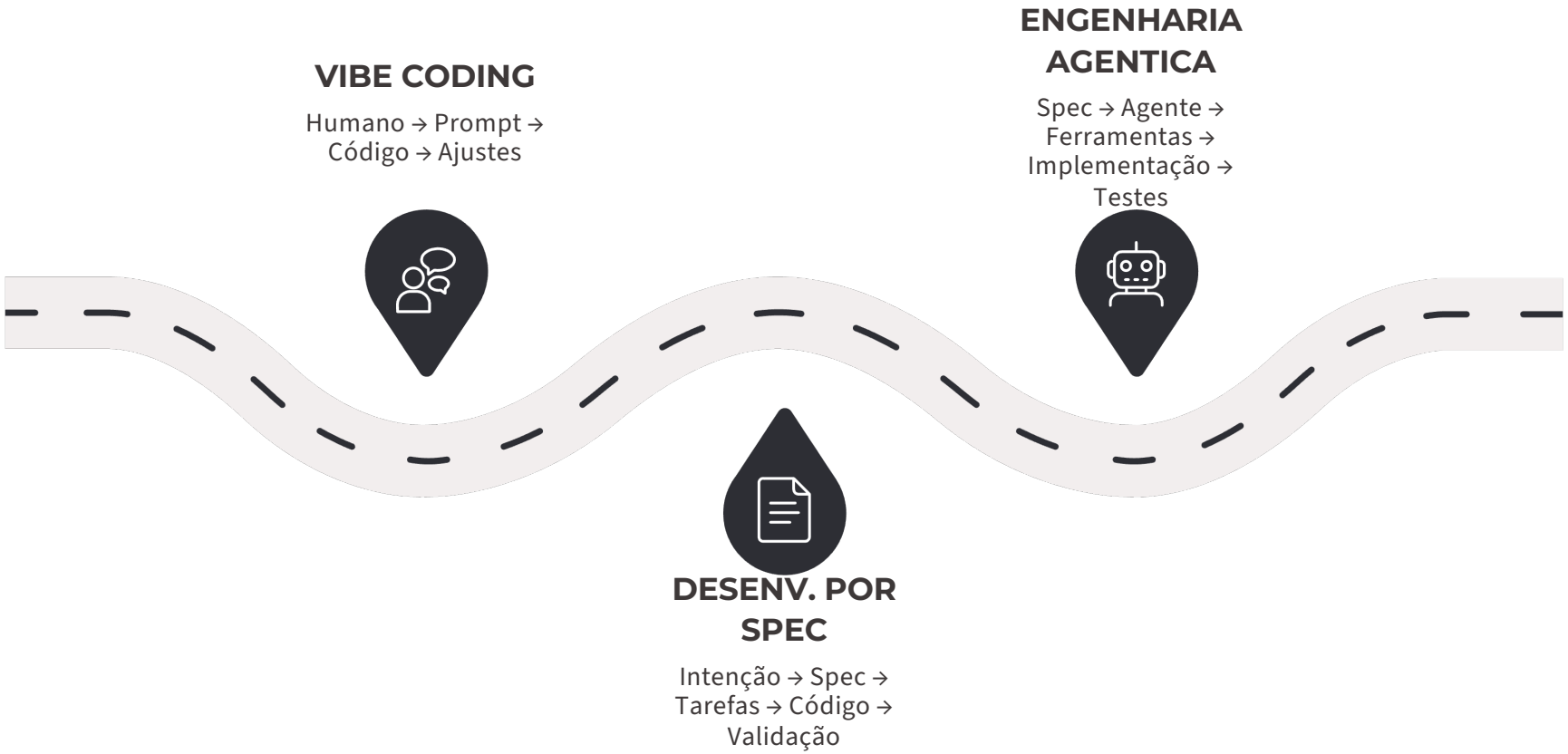
Agentic Engineering responde:

COMO permitir que agentes executem isso?

- Contexto
- Ferramentas
- Agentes
- Guardrails
- Evals
- Feedback loops

Eles são complementares. SDD dá direção ao agente. Agentic Engineering cria o ambiente para o agente trabalhar.

A evolução do desenvolvimento com IA



> "O futuro não é apenas programar com IA. É projetar o contexto, as especificações e os sistemas que permitem à IA programar corretamente."